

Scott-Toolkit v5.2 — How to Use It

This is your personal cheat sheet. The toolkit handles context engineering -- making sure Claude Code always knows where you left off, what you're building, and what lessons you've learned. It also enforces stack-specific correctness, detects plugin-project misalignment, and feeds lessons back into reusable checks. It works alongside GSD (project management) and Superpowers (dev methodology).

Starting a Session

Just open Claude Code in your project folder. The toolkit handles the rest:

- 1. **Session-start hook fires automatically** — it checks for context files and shows you what's available
- 2. **If you see "Resume file found"** — Claude will read `.claude-resume.md` and know exactly where you left off
- 3. **If you see "No resume file"** — type `/scott:resume-project` for a guided walkthrough of getting back up to speed

If you're in your home directory (~), the hook shows your active projects list. Pick one and `cd` into it.

The Slash Commands

These are your workflows. Type them and Claude walks you through each one.

Project Workflows

Command	When to use it
<code>/scott:new-feature</code>	Add a feature to the current project with guided workflow
<code>/scott:new-project</code>	Start a new project with the guided 8-phase workflow
<code>/scott:phase-closeout</code>	Run the mandatory phase closeout (verify, review, reflect, gate)
<code>/scott:resume</code>	Pick up where you left off on a project

Stack Enforcement & Learning

Command	When to use it
<code>/scott:stack-baseline</code>	First-run stack audit for existing projects
<code>/scott:stack-review</code>	Monthly stack health dashboard (check metrics, learning loop)

Learning & Improvement

Command	When to use it
<code>/scott:compare-sources</code>	Compare new sources against your toolkit and surface what's actionable
<code>/scott:update-toolkit</code>	Update the scott-toolkit with new lessons or patterns

Reference & Knowledge

Command	When to use it
<code>/scott:debug</code>	Debug a problem with structured diagnosis
<code>/scott:n8n-reference</code>	Complete n8n workflow reference (patterns, code nodes, expressions, validation)
<code>/scott:surrealdb</code>	SurrealDB query patterns, schema design, AI agent architecture, and connection setup

Tools & Utilities

Command	When to use it
/scott:pause	Generate a resume prompt for picking up later
/scott:toolkit-briefing	Deep-read the toolkit so Claude fully understands it

Business Context (Advosy)

Command	When to use it
/advosy:context	Get Advosy company context (leadership, subsidiaries, contacts)
/advosy:claimsforce	Get Claimsforce automation context (workflows, placeholders, webhooks)
/advosy:crm	CRM design system, mockup patterns, and page prototyping context

You don't need to memorize these. Just describe what you want to do and Claude will suggest the right one.

What Happens Automatically (You Don't Need to Do Anything)

When you start a session

- Hook scans for project context files (CLAUDE.md, PRD.md, todo.md, resume file)
- Hook rebuilds ~/Sites/Global/ACTIVE-PROJECTS.md from all your project resume files
- Hook checks plugin-project alignment (e.g., warns if Vercel plugin is active on a non-Vercel project)
- You see a short summary of what's available

When context is about to compact

- Hook saves a mechanical backup (.context-snapshot.md)
- Hook tells Claude to write a proper resume file (.claude-resume.md)
- Hook prompts Claude to capture any notable session patterns to ~/.claude/instinct-candidates.md
- **You might see Claude write a file right before compaction — that's normal and intentional**
- **After compaction**, Claude re-reads the resume file, snapshot, current task, and any offloaded files before continuing

When a session ends

- Hook reminds Claude to write the resume file and update docs
- Hook prompts Claude to capture any notable session patterns to ~/.claude/instinct-candidates.md
- This is your safety net — even if you forget, the reminder fires

Guard hooks (always running)

- **git push** — Claude only pushes when you explicitly ask; Claude Code's permission prompt is the safety net
 - **Destructive commands blocked** — `rm -rf`, `git reset --hard`, etc. require explanation
 - **CLAUDE.md protected** — can't be overwritten without confirmation
 - **npm install blocked** — new dependencies need your approval
-

Pausing Work (The Most Important Part)

When you're done for the day or switching projects, you have two options:

Option A: Just close Claude Code

The session-end hook will remind Claude to write the resume file. If Claude has time before the session closes, it writes `.claude-resume.md` with everything needed to continue.

Option B: Tell Claude "let's stop here"

Claude will:

1. Write/update `.claude-resume.md` with current state
2. Update CLAUDE.md's Current Status section
3. Mark completed tasks in `tasks/todo.md`

Option B is better because Claude has full context when you explicitly pause. If you just close the window, the hook fires but Claude might not have time to respond.

If context fills up mid-session

The PreCompact hook fires automatically. Claude writes the resume file before compaction compresses the conversation. After compaction, Claude reads the resume file back and continues. You shouldn't notice any disruption.

The Resume File (`.claude-resume.md`)

This is the magic. It's a ~10-line file in your project root that tells the next session exactly where to pick up.

You never write this file. Claude writes it. It contains:

- What workflow/phase you were in
- What's done and what's next
- Key decisions made during the session

You never read this file manually either. Claude reads it at session start. It's a machine-to-machine handoff note.

If you're curious, it looks like this:

```
# Resume Context
Workflow: /scott:new-feature – Phase 4 (Build)
PM Mode: GSD
Branch: feat/user-auth

Done: Mini-PRD approved. Schema updated.
Current: Auth middleware written, not connected to routes.
Next: Connect middleware, then verify.
Decisions: JWT over sessions. 15min access / 7day refresh.
```

Active Projects List (`~/Sites/Global/ACTIVE-PROJECTS.md`)

This file is auto-generated every time you start a session. It lists all projects that have a `.claude-resume.md` file. You can glance at it to see what's in flight:

Project	Last Worked	Status	Path
life-os	2026-03-01	Phase 3: data model	~/Sites/life-os
advosy-crm	2026-02-28	Feature: user auth	~/Sites/advosy-crm

Don't edit this file. It gets rebuilt from resume files every session. When a project is done and its resume file is removed, it disappears from this list automatically.

Multi-Machine Sync

The toolkit lives at `~/Sites/Global/scott-toolkit/` and is a git repo. To sync across machines:

```
# On any machine – pull latest
cd ~/Sites/Global/scott-toolkit && git pull

# If hooks/rules/skills seem off – re-deploy
./setup.sh
```

The setup script uses symlinks, so after `git pull` most changes take effect immediately without re-running setup. Re-run setup only if new hooks or skills were added.

Quick Reference: What Lives Where

Location	What's there	Who maintains it
<code>~/Sites/Global/scott-toolkit/</code>	The toolkit repo (source of truth)	You + Claude via <code>/scott:update-toolkit</code>
<code>~/ .claude/hooks/</code>	Deployed hooks (symlinked to toolkit)	<code>setup.sh</code>
<code>~/ .claude/rules/</code>	Behavior rules (symlinked to toolkit)	<code>setup.sh</code>
<code>~/ .claude/skills/scott-*/</code>	Deployed workflow skills	<code>setup.sh</code>
<code>~/ .claude/checks/</code>	Stack enforcement check files	<code>setup.sh</code>
<code>~/ .claude/tools/</code>	CLI tools (stack-detect, stack-check, etc.)	<code>setup.sh</code>
<code>~/ .claude/config/</code>	Toolkit config (interfaces.json)	<code>setup.sh</code>
<code>~/ .claude/settings.json</code>	Hook registrations, global plugin toggles	Manual (machine-specific)
<code><project>/ .claude/settings.json</code>	Per-project plugin overrides (e.g., disable Vercel)	<code>/scott:new-project</code> or session-start suggestion
<code><project>/stack-lock.json</code>	Locked technology versions for that project	<code>/scott:new-project</code> or manual
<code><project>/ .claude-resume.md</code>	Resume file for that project	Claude (automatically)
<code>~/Sites/Global/ACTIVE-PROJECTS.md</code>	Master project list	Session-start hook (automatically)

What Runs Without You Thinking About It

Everything below happens behind the scenes. You'll never need to trigger, configure, or maintain any of it. It's listed here so you know what's going on if you see something unexpected.

Files that write themselves

File	What it is	When it's created/updated	By whom
<code>.claude-resume.md</code>	Per-project handoff note (~10 lines)	Before compaction, at session end, at workflow end	Claude
<code>.context-snapshot.md</code>	Mechanical backup (git state, todos)	Right before compaction	Hook (shell script)
<code>~/Sites/Global/ACTIVE-PROJECTS.md</code>	Master list of all in-flight projects	Every session start	Hook (shell script)
<code>.claude/tool-output-overflow/*.md</code>	Offloaded large tool results (>4KB)	When a tool returns a large result	Hook (shell script)

File	What it is	When it's created/updated	By whom
<code>~/ .claude/instinct-candidates.md</code>	Auto-captured session patterns for review	Before compaction + session end	Claude (prompted by hook)

You never create, edit, or delete these files. They're machine-to-machine communication.

Hooks that fire on their own

Hook	When it fires	What it does	What you see
session-start.sh	Every time you open Claude Code	Scans for resume files, rebuilds active projects list, checks plugin-project alignment, tells Claude what context exists	A 2-3 line message at the top of your session (plus plugin mismatch warning if applicable)
pre-compact.sh	When the context window is about to compress	Saves mechanical backup, tells Claude to write the resume file immediately	Claude suddenly writing a file — that's normal
session-end.sh	When Claude Code is closing	Reminds Claude to write resume file + update docs	A short checklist reminder
guard-git-push.sh	<i>(retired v6.2.1)</i>	No longer blocks; Claude Code's permission prompt gates pushes	N/A
guard-destructive.sh	Every time Claude tries <code>rm -rf, git reset --hard</code> , etc.	Blocks it and explains why	"Blocked: [command] would..." message
guard-claude-md.sh	Every time Claude tries to edit <code>CLAUDE.md</code> or <code>MEMORY.md</code>	Blocks it until you confirm	"CLAUDE.md modification blocked" message
guard-npm-install.sh	Every time Claude tries to install packages	Blocks it and lists the packages	"npm install blocked -- packages: ..." message
guard-phase-completion.sh	When GSD tries to mark a phase complete	Blocks without <code>.post-execution-complete</code> marker	"Phase closeout required" message
check-file-test-trigger.sh	When a check file in <code>checks/</code> is edited	Auto-runs test fixtures against the changed check	Test pass/fail results
version-propagate.sh	When <code>CHANGELOG.md</code> in the toolkit is edited	Checks all files in <code>version-manifest.json</code> for stale version references	Bordered checklist of files needing updates (or nothing if all current)
uiux-reminder.sh	After <code>.vue</code> files are written during GSD execution	One-line nudge to run <code>/impeccable:audit</code> before closeout	"Consider running <code>/impeccable:audit</code> "
offload-large-output.sh	After every tool use	Writes tool results >4KB to <code>.claude/tool-output-overflow/</code> to prevent context bloat	"Large tool output saved to..." message
extract-instincts.sh	Before compaction + session end	Prompts Claude to note session patterns to <code>~/ .claude/instinct-candidates.md</code>	Claude may write a quick pattern note
pre-completion-checklist.sh	When a session ends	Checks for uncommitted changes, stale <code>todo.md</code> , missing resume file, and stale <code>lessons.md</code> . Strong visual warning but doesn't block.	A bordered checklist with item count
auto-format.sh	After every Write or Edit (<code>.js/.ts/.vue/.css/.json</code>)	Runs Prettier if the project has it configured. Uses local binary for speed. Notifies Claude when formatting changes occur.	"Auto-formatted [file] with Prettier" (or nothing if no Prettier)
context-reminders.sh	After every tool use	Tracks session duration and tool count via <code>/tmp</code> state file. Warns at 60 min and 100 tool uses. Replaces behavioral self-monitoring.	"Context Health Check" box with suggestions
bash logger	Every Bash command Claude runs	Logs the command to <code>~/ .claude/bash-commands.log</code>	Nothing — completely silent

Hook	When it fires	What it does	What you see
GSD context monitor	After every tool use	Tracks how full the context window is	Warning when context reaches 80%+
GSD update checker	At session start	Checks if GSD has updates available	Update notification if one exists
GSD status line	Continuously	Shows project/phase info in the status bar	A status line at the bottom of your terminal
Notification sound	When Claude needs your attention	Plays a sound + macOS notification	"Blow" sound + popup

Behavior rules always loaded

These rules live in `~/ .claude/rules/` and Claude reads them automatically in every session, every project. You never need to reference them.

Rule	What it tells Claude
claude-behavior.md	Abstract operation resolution via interfaces.json. Stack enforcement (stack-lock.json, stack-check.sh, drift detection). Use Superpowers for dev methodology, GSD for project management, toolkit for learning capture. Pre-completion verification gate. Task contracts. Doom-loop detection. Named agent roles. Checkpoint commits. Context rot monitoring. Post-compaction recovery. Neutral prompting.
code-style.md	TypeScript strict mode, Vue 3 Composition API, Tailwind v4, Pinia, 2-space indent, single quotes.
n8n-sync.md	Keep local tools-needing-setup file and n8n reminder workflow in sync.

Symlinks (why updates "just work")

The hooks and rules in `~/ .claude/` are symlinks — shortcuts that point back to `~/Sites/Global/scott-toolkit/`. When you `git pull` toolkit updates, the symlinks still point to the same files, so the updates take effect instantly. No re-deploy needed unless brand new files were added.

Native Claude Code Commands

These are built-in — no setup needed. Just type them.

Session Management

Command	What it does
/clear	Clear conversation and free context
/compact [instructions]	Compress conversation (optional focus instructions)
/resume	Resume a previous session (interactive picker)
/fork [name]	Fork conversation at this point — try two approaches safely
/rename [name]	Name current session for easy finding later
/export [filename]	Save conversation as plain text
/rewind	Rewind code and/or conversation to a checkpoint (Esc+Esc shortcut)

Code & Git

Command	What it does
/diff	Interactive diff viewer for uncommitted changes
/review	Review a PR for quality, correctness, security, tests
/security-review	Scan pending changes for security vulnerabilities

Command	What it does
/pr-comments [PR]	Fetch and display comments from a GitHub PR
/plan	Enter plan mode (read-only analysis, then propose changes)

Model & Output

Command	What it does
/model [model]	Change AI model (sonnet, opus, haiku)
`/fast [on off]	
/output-style [style]	Switch between Default, Explanatory, Learning styles

Configuration

Command	What it does
/config	Open settings interface
/permissions	View or update tool permissions
/memory	Edit CLAUDE.md files and auto-memory
/hooks	Manage hook configurations
/plugin	Manage plugins
/keybindings	Configure keyboard shortcuts
/statusline	Configure the status line display
/theme	Change color theme
/vim	Toggle vim editing mode

Tools & Integration

Command	What it does
/mcp	Manage MCP server connections
/ide	Manage IDE integrations
/chrome	Configure Chrome browser automation
/add-dir <path>	Add a working directory to current session
/skills	List all available skills
/tasks	List and manage background tasks

Info & Diagnostics

Command	What it does
/help	Show help and available commands
/cost	Token usage and spending stats
/usage	Plan limits and rate limit status
/context	Visual grid of context window usage
/stats	Session history, streaks, usage patterns
/insights	Generate report analyzing your Claude Code sessions
/doctor	Diagnose installation issues
/release-notes	View changelog
/status	Version, model, and account info

Useful Keyboard Shortcuts

Shortcut	What it does
Esc + Esc	Rewind/summarize (same as /rewind)
Shift+Tab	Cycle permission modes (normal → plan → auto)
Ctrl+V	Paste image from clipboard
Ctrl+T	Toggle task list
Ctrl+O	Toggle verbose output (see Claude's thinking)
Ctrl+B	Background a running task
@	File path autocomplete (type @ then start typing)
!	Bash mode (run command directly)
?	Show available shortcuts

Active Features

Phase Auto-Advancement

Every workflow phase is tagged with one of three behaviors:

Tag	Meaning	What Claude Does
[STOP]	Judgment phase — needs your input/approval	Pauses and waits for you
[AUTO]	Mechanical phase — output is deterministic	Shows a one-line summary, proceeds immediately
[DELEGATE]	Hands off to GSD/Superpowers	Shows what was delegated, proceeds

No tag defaults to [STOP] (safe default). AUTO means "don't wait for permission," not "skip" — if an AUTO phase hits something unexpected, Claude will still pause and ask.

PM Mode: GSD + Superpowers Integration Model

All projects use **GSD + Superpowers** together. They have distinct roles:

GSD = orchestration engine (what to do, when, and tracking progress):

- Phase planning: **plan_phase** operation
- Phase execution: **execute_phase** operation
- Verification: **verify_work** operation
- State tracking: `.planning/` directory (STATE.md, ROADMAP.md, phase plans)
- Quick tasks: **quick_task** operation
- Test coverage: **add_tests** operation

Superpowers = discipline layer (how to do it well):

- Git isolation: **git_worktree** operation (feature branches in worktrees)
- TDD enforcement: **tdd** operation (failing test, implement, refactor)
- Code review: **code_review** operation
- Plan quality: **write_plan** operation (for non-GSD contexts only)
- Branch completion: **finish_branch** operation (merge or PR)

Operation names (like `plan_phase`, `tdd`) resolve to concrete commands via `~/Sites/Global/scott-toolkit/config/interfaces.json`. This means if GSD or Superpowers renames a command, only one file changes.

The build loop (how they work together):

1. Worktree -> isolation for GSD execution
2. Plan -> structured task breakdown
3. Execute -> TDD discipline applies, `stack-check.sh` runs on changed files

4. Review -> after GSD execution completes
5. Verify -> UAT against acceptance criteria
6. Finish branch -> merge or PR

Proactive Toolkit Usage

Claude suggests the next skill automatically after completing each one:

Just finished	What Claude suggests next
/scott:new-project (build)	Phase closeout -> Finish branch
/scott:new-feature (build)	Phase closeout -> Finish branch
/scott:debug (fix applied)	Resume interrupted work
/scott:phase-closeout	Update toolkit if improvements found
/gsd:execute-phase	Phase closeout (auto-invoked, hook-enforced)
/gsd:verify-work (passes)	Finish branch
Code review passes	Finish branch
Branch merged	Phase closeout if milestone complete
Session wrapping up	Pause

Claude also invokes skills proactively when it detects triggers (bugs, satisfaction, errors, session end) without being asked.